

break e continue

break e continue

Come controllare il flusso dei cicli con break e continue in C++.

Controllare il flusso dentro un ciclo

A volte, mentre stai scorrendo una lista, vuoi fermarti non appena hai trovato quello che cercavi. Oppure vuoi saltare certi elementi e andare direttamente al prossimo.

`break` e `continue` sono due istruzioni che ti danno questo controllo preciso sul comportamento dei cicli.

break : esci dal ciclo

`break` interrompe immediatamente il ciclo. L'esecuzione continua con la prima istruzione dopo il ciclo, come se il ciclo fosse finito.

```
for (int i = 0; i < 10; i++) {  
    if (i == 5) {  
        break; // usciamo quando i vale 5  
    }  
    cout << i << " ";  
}  
cout << "fine";  
// Output: 0 1 2 3 4 fine
```

Quando usare **break**

Il caso più comune: cercare qualcosa in una lista. Appena lo trovi, non ha senso continuare a cercare:

```

int numeri[] = {10, 25, 3, 47, 8, 15};
int dimensione = 6;
int daQuerare = 47;
bool trovato = false;

for (int i = 0; i < dimensione; i++) {
    if (numeri[i] == daQuerare) {
        cout << "Trovato " << daQuerare << " alla posizione " << i << endl;
        trovato = true;
        break; // trovato! inutile continuare a cercare
    }
}

if (!trovato) {
    cout << daQuerare << " non trovato." << endl;
}

```

break nei cicli annidati

break esce solo dal ciclo più interno in cui si trova. Il ciclo esterno continua normalmente:

```

for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        if (j == 1) {
            break; // esce solo dal ciclo interno (j)
        }
        cout << "(" << i << "," << j << ") ";
    }
    cout << endl;
}

```

Output:

```

(0,0)
(1,0)
(2,0)

```

continue : salta al prossimo giro

`continue` non esce dal ciclo. Salta solo il resto del blocco corrente e passa direttamente alla prossima iterazione.

```
for (int i = 0; i < 10; i++) {  
    if (i % 2 == 0) {  
        continue; // se il numero è pari, saltiamo questa iterazione  
    }  
    cout << i << " "; // stampiamo solo i dispari  
}  
// Output: 1 3 5 7 9
```

Quando usare `continue`

Utile per saltare certi casi senza dover annidare tutto il codice dentro un `if` :

```
// Stampa solo i numeri non divisibili per 3  
for (int i = 1; i <= 20; i++) {  
    if (i % 3 == 0) {  
        continue; // saltiamo i multipli di 3  
    }  
    cout << i << " ";  
}  
// Output: 1 2 4 5 7 8 10 11 13 14 16 17 19 20
```

Un altro esempio: ignorare i dati non validi e continuare con quelli validi:

```

int voti[] = {8, -1, 7, 150, 6, 9};
int n = 6;
int somma = 0;
int contatore = 0;

for (int i = 0; i < n; i++) {
    if (voti[i] < 0 || voti[i] > 10) {
        cout << "Voto non valido ignorato: " << voti[i] << endl;
        continue; // saltiamo questo voto e passiamo al prossimo
    }
    somma += voti[i];
    contatore++;
}

if (contatore > 0) {
    cout << "Media voti validi: " << (double)somma / contatore << endl;
}

```

break e continue nel while

Funzionano allo stesso modo anche in `while` e `do...while` :

```

// Ciclo "infinito" controllato da break
int i = 0;
while (true) {
    if (i >= 5) {
        break; // uscita manuale dal ciclo
    }
    cout << i << " ";
    i++;
}
// Output: 0 1 2 3 4

```

```
int i = 0;
while (i < 10) {
    i++;
    if (i == 5) {
        continue; // salta il 5
    }
    cout << i << " ";
}
// Output: 1 2 3 4 6 7 8 9 10
```

Consigli pratici

- Usa `break` e `continue` con moderazione. Troppi rendono il codice difficile da seguire.
- Se un ciclo ha molti `break`, potrebbe essere riscrivibile con una condizione più precisa:

```
// Con break (meno chiaro)
for (int i = 0; i < 100; i++) {
    if (i * i > 50) {
        break;
    }
    cout << i << " ";
}

// Alternativa più chiara: la condizione di uscita è nella testa del for
for (int i = 0; i < 100 && i * i <= 50; i++) {
    cout << i << " ";
}
```

- Se usi `break` o `continue` per un motivo non ovvio, aggiungi un commento che spiega il perché.